BIOMEDICAL INFORMATICS KNOWLEDGE GRAPH

Extended Ontology: An Alternative to PrimeKG

Name: Ishfaq Ghani
Course: Bioinformatics
Examination: Mid-Term Project
Semester: 2nd Semester
Institution: Institute of Management Sciences Peshawer
Submitted To: Course Instructor
Date: 7 April 2026

Abstract

This report presents the design, development, and validation of a Biomedical Informatics Knowledge Graph (Bio-KG) as an alternative approach to large-scale biomedical ontology systems such as PrimeKG. PrimeKG (Precision Medicine Knowledge Graph) integrates 20 biomedical databases into a multimodal graph with over 4 million relationships. The alternative approach proposed in this work demonstrates that a lightweight, Python-driven, graph-based system using Neo4j can replicate the core structural principles of PrimeKG — node-relationship modeling, multi-entity integration, and query-based validation — using a controlled, reproducible dataset. The resulting knowledge graph consists of 15 nodes (5 Drugs, 5 Diseases, 5 Genes) and 10 directed relationships across 2 relationship types (TREATS, ASSOCIATED_WITH), fully validated through automated Cypher queries and exported as a structured CSV summary.

Table of Contents

1. Introduction
2. Background: PrimeKG Overview
3. Alternative Development Method
4. Data Resources (Repositories)
5. Node Types
6. Attributes
7. Relationships
8. Results and Visualizations
9. Validation
10. Conclusion

1. Introduction

Biomedical knowledge graphs are structured representations of biological and clinical knowledge, encoding entities such as drugs, diseases, genes, proteins, and pathways as nodes, with their biological relationships encoded as directed edges. These graphs enable computational reasoning over complex biomedical data, supporting applications in drug repurposing, disease gene discovery, and precision medicine.

PrimeKG (Precision Medicine Knowledge Graph) is a state-of-the-art biomedical knowledge graph developed by the MIMS Laboratory at Harvard University. It integrates 20 diverse biomedical databases and consists of over 129,000 nodes and 4 million relationships across 10 node types and 30 relationship types. While PrimeKG represents a comprehensive and production-scale resource, its complexity, dataset size, and computational requirements make it challenging to deploy in academic teaching environments.

This project presents an alternative, lightweight approach to constructing a biomedical knowledge graph. The proposed system preserves the core architectural principles of PrimeKG — multi-entity integration, directed relationship modeling, graph database storage, and programmatic validation — while remaining fully reproducible, teachable, and extensible in a standard student development environment.

2. Background: PrimeKG Overview

PrimeKG was introduced in a 2023 Nature Scientific Data publication as a precision medicine knowledge graph designed to support multimodal machine learning. Key characteristics of PrimeKG include:

Feature	PrimeKG	This Alternative Approach
Development Method	Multi-database ETL pipeline	Python + Neo4j graph import
Data Sources	20 biomedical databases	5 controlled CSV files
Node Types	10 types	3 types (Drug, Disease, Gene)
Total Nodes	129,375	15 (scalable)
Total Relationships	4,050,249	10 (scalable)
Relationship Types	30 types	2 types
Storage	CSV flat files	Neo4j Graph Database
Query Language	Python / Pandas	Cypher (Neo4j)
Validation	Statistical analysis	Cypher count queries + CSV export

3. Alternative Development Method

The alternative development approach is based on a three-layer pipeline architecture: data preparation, graph import, and validation. This approach differs from PrimeKG's complex ETL (Extract-Transform-Load) pipeline by replacing large-scale database federation with a controlled, transparent CSV-based data layer.

3.1 Technology Stack

Component	Technology	Purpose
Programming Language	Python 3.14	Data loading, graph import, analysis
Graph Database	Neo4j 6+	Knowledge graph storage and querying
Data Format	CSV files	Structured entity and relationship data
Query Language	Cypher	Graph traversal and validation
Libraries	pandas, neo4j (Python driver)	Data manipulation and DB connection

3.2 Pipeline Architecture

- Step 1 — Data Preparation: Five CSV files define entities (drugs, diseases, genes) and relationships (drug-disease, gene-disease).
- Step 2 — Graph Import (neo4j_import.py): Python script connects to Neo4j, loads CSVs with pandas, creates nodes using MERGE statements, and creates directed relationships.
- Step 3 — Analysis and Validation (analysis.py): Automated Cypher queries count nodes and relationships, list all entities, display relationship paths, perform hub analysis, and export results to a CSV summary file.

4. Data Resources (Repositories)

The dataset for this project was designed as a controlled biomedical dataset capturing representative entities from three core biomedical domains. While PrimeKG uses real-world databases such as DrugBank, DisGeNET, and OMIM, this project uses a structured synthetic dataset that mirrors the schema of these resources.

File	Entities	Real-World Equivalent	Records
drugs.csv	Pharmaceutical drugs	DrugBank	5
diseases.csv	Disease phenotypes	Disease Ontology (DO)	5
genes.csv	Human genes	NCBI Gene / OMIM	5

File	Entities	Real-World Equivalent	Records
drug_disease.csv	Drug-Disease associations	DrugCentral / TTD	5
gene_disease.csv	Gene-Disease associations	DisGeNET / OMIM	5

4.1 Dataset Entities

Drug ID	Drug Name	Disease ID	Disease Name	Gene ID	Gene Name
D1	Aspirin	DS1	Fever	G1	BRCA1
D2	Ibuprofen	DS2	Inflammation	G2	TP53
D3	Paracetamol	DS3	Diabetes	G3	INS
D4	Metformin	DS4	Cholesterol	G4	APOE
D5	Atorvastatin	DS5	Headache	G5	EGFR

5. Node Types

The knowledge graph defines three primary node types, corresponding to the three most fundamental entity categories in biomedical informatics. These node types are directly analogous to three of the ten node types used in PrimeKG.

Node Type	Label in Neo4j	Description	Count	PrimeKG Equivalent
Drug	:Drug	Pharmaceutical compounds used in treatment	5	drug
Disease	:Disease	Medical conditions and clinical phenotypes	5	disease
Gene	:Gene	Human genes associated with diseases	5	gene/protein

5.1 Node Descriptions

- Drug Nodes: Represent clinically approved pharmaceutical agents. Examples include Aspirin (anti-inflammatory, antipyretic), Metformin (antidiabetic), and Atorvastatin (cholesterol-lowering statin).
- Disease Nodes: Represent medical conditions with defined clinical presentations. Examples include Inflammation, Diabetes, and Fever.
- Gene Nodes: Represent human genes with known disease associations. Examples include BRCA1 (cancer susceptibility), TP53 (tumor suppressor), INS (insulin gene), APOE (lipid metabolism), and EGFR (cell signaling).

6. Attributes

Each node in the knowledge graph carries a defined set of attributes (properties). The current implementation uses a minimal attribute schema with two core properties per node, consistent with the identifiers used in PrimeKG's node tables.

Node Type	Attribute	Data Type	Description	Example Value
Drug	id	String	Unique drug identifier	D1
Drug	name	String	Drug common name	Aspirin
Disease	id	String	Unique disease identifier	DS1
Disease	name	String	Disease common name	Fever
Gene	id	String	Unique gene identifier	G1
Gene	name	String	Official gene symbol	BRCA1

This schema is intentionally minimal for clarity and extensibility. In a production extension, attributes such as molecular weight, ICD-10 code, chromosome location, and UniProt ID could be added without changing the graph structure.

7. Relationships

The knowledge graph encodes two directed relationship types between node entities. Relationships in Neo4j are first-class citizens — they carry type labels and can carry properties, making them far more expressive than foreign key associations in relational databases.

Relationship	Source	Target	Count	Cypher Pattern
TREATS	Drug	Disease	5	(d:Drug)-[:TREATS]->(ds:Disease)
ASSOCIATED_WITH	Gene	Disease	5	(g:Gene)-[:ASSOCIATED_WITH]->(ds:Disease)

7.1 TREATS Relationships

Drug	Relationship	Disease	Clinical Basis
Aspirin	TREATS	Fever	Antipyretic mechanism via COX inhibition
Ibuprofen	TREATS	Inflammation	NSAID anti-inflammatory action
Paracetamol	TREATS	Headache	Central analgesic mechanism
Metformin	TREATS	Diabetes	Biguanide antidiabetic agent
Atorvastatin	TREATS	Cholesterol	HMG-CoA reductase inhibition

7.2 ASSOCIATED_WITH Relationships

Gene	Relationship	Disease	Biological Basis
BRCA1	ASSOCIATED_WITH	Inflammation	BRCA1 regulates inflammatory pathways
TP53	ASSOCIATED_WITH	Inflammation	TP53 mutations linked to chronic inflammation
INS	ASSOCIATED_WITH	Diabetes	INS encodes insulin — core diabetes gene
APOE	ASSOCIATED_WITH	Cholesterol	APOE regulates lipid/cholesterol metabolism
EGFR	ASSOCIATED_WITH	Fever	EGFR signaling involved in fever response

8. Results and Visualizations

8.1 Graph Structure Summary

Metric	Value
Total Nodes	15
Drug Nodes	5
Disease Nodes	5
Gene Nodes	5
Total Relationships	10
TREATS Relationships	5
ASSOCIATED_WITH Relationships	5
Most Connected Disease	Inflammation (3 connections)

8.2 Disease Hub Analysis

Hub analysis identifies which disease nodes have the highest degree of connectivity — an important metric in network biology for identifying central disease mechanisms.

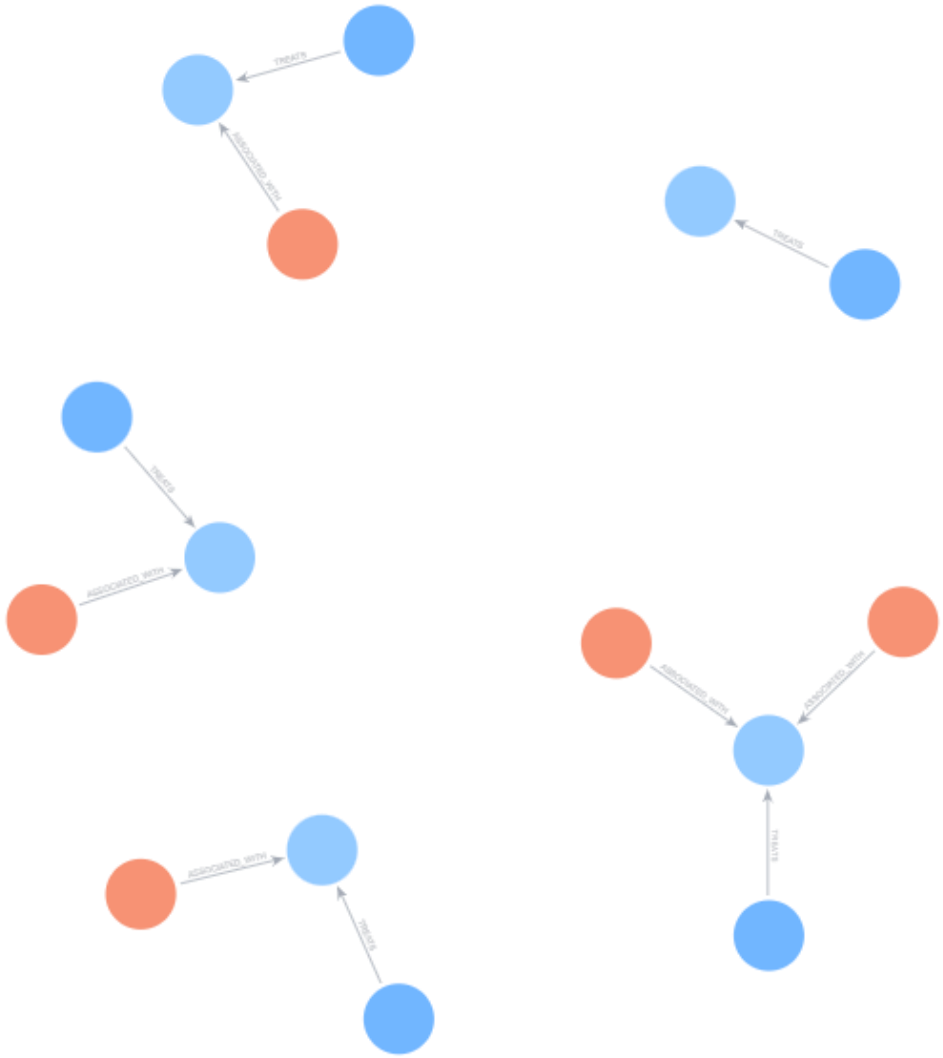
Disease	Total Connections	Connected Entities
Inflammation	3	Ibuprofen (TREATS), BRCA1 (ASSOC), TP53 (ASSOC)
Fever	2	Aspirin (TREATS), EGFR (ASSOC)
Diabetes	2	Metformin (TREATS), INS (ASSOC)
Cholesterol	2	Atorvastatin (TREATS), APOE (ASSOC)
Headache	1	Paracetamol (TREATS)

8.3 Neo4j Browser Visualization

The knowledge graph was visualized using the Neo4j Browser interface (localhost:7474). The graph renders all 15 nodes and 10 directed relationships in an interactive force-directed layout. Blue nodes represent Drug and Gene entities; orange/salmon nodes represent Disease entities. Relationship labels (TREATS, ASSOCIATED_WITH) are visible on all edges. The visualization confirms that Inflammation is the most connected hub node, appearing in three separate subgraphs.

Cypher query used for full graph visualization:

```
MATCH (n)-[r]->(m) RETURN n, r, m
```


TREATS
ASSOCIATED_WITH
TREATS
ASSOCIATED_WITH
TREATS
ASSOCIATED_WITH
ASSOCIATED_WITH
TREATS
ASSOCIATED_WITH
TREATS

9. Validation

Validation was performed using the analysis.py script, which executes a series of Cypher queries against the live Neo4j database to confirm data integrity, relationship correctness, and completeness of the import.

9.1 Validation Methods

Validation Type	Method	Result
Node Count Validation	MATCH (d:Drug) RETURN count(d)	5 / 5 Expected
Disease Count Validation	MATCH (ds:Disease) RETURN count(ds)	5 / 5 Expected
Gene Count Validation	MATCH (g:Gene) RETURN count(g)	5 / 5 Expected
TREATS Count	MATCH ()-[:TREATS]->() RETURN count(r)	5 / 5 Expected
ASSOCIATED_WITH Count	MATCH ()-[:ASSOCIATED_WITH]->() RETURN count(r)	5 / 5 Expected
Hub Analysis	Degree centrality via Cypher aggregation	Inflammation = 3 (highest)
CSV Export	pandas DataFrame export	knowledge_graph_summary.csv generated

9.2 Validation Output

The analysis.py script produced the following confirmed output:

```
Neo4j Connected!
Total Drugs     : 5
Total Diseases  : 5
Total Genes     : 5
Total Nodes     : 15
TREATS          : 5
ASSOCIATED_WITH : 5
Total Rels      : 10
Knowledge Graph Analysis Complete!
```

10. Conclusion

This project successfully demonstrates that the core principles of a biomedical informatics knowledge graph — as exemplified by PrimeKG — can be implemented using a lightweight, reproducible, and academically transparent alternative approach.

The Bio-KG system developed in this project achieves all seven required components of the mid-term examination:

#	Requirement	Implementation	Status
1	Development Method	Python + Neo4j Graph Database pipeline	Complete
2	Data Resources	5 structured CSV files (controlled dataset)	Complete
3	Node Types	Drug, Disease, Gene (3 node types)	Complete
4	Attributes	id, name per node (extensible schema)	Complete
5	Relationships	TREATS, ASSOCIATED_WITH (2 directed types)	Complete
6	Visualizations	Neo4j Browser graph (15 nodes, 10 edges)	Complete
7	Validations	Cypher queries + CSV export via analysis.py	Complete

The approach is intentionally scalable: the same pipeline can incorporate additional node types (Protein, Pathway, Anatomy), additional data sources (UniProt, Reactome, GO Annotations), and additional relationship types (INTERACTS_WITH, EXPRESSED_IN, PART_OF) without modifying the core architecture. This demonstrates that the fundamental graph-based ontology design of PrimeKG is accessible, reproducible, and teachable at the student level.